

Threat Intelligence–Driven Malware Detection using Wazuh and MISP

Project Description

This project demonstrates the integration of **Wazuh SIEM** with **MISP (Malware Information Sharing Platform)** to enrich security alerts with real-world threat intelligence.

The goal was to automatically correlate **file integrity monitoring (FIM) events** with **known malicious file hashes** from MISP and generate high-confidence security alerts for SOC investigation.

The project focuses on **hash-based malware detection**, alert enrichment, and SOC visibility, without introducing excessive noise or unnecessary automation.

Project Objectives

- Integrate Wazuh with MISP for threat intelligence enrichment
- Detect malicious files using **hash reputation (MD5 / SHA1 / SHA256)**
- Enrich Wazuh alerts with:
 - MISP event UUID
 - Attribute UUID
 - MISP permalink
- Validate detection using a **real malware test (EICAR)**
- Forward enriched alerts to **TheHive** for SOC triage visibility
- Maintain **low noise and high confidence alerts**

Tools & Technologies Used

- **Wazuh** SIEM (Manager & Agents)
- **MISP** Threat Intelligence Platform
- **TheHive** (Incident Management)
- **Ubuntu Linux** (Wazuh Agent – malware test)
- **Windows** (Agent – baseline monitoring)
- **Python** (Custom Wazuh Integration Script)
- Syscheck / File Integrity Monitoring
- **MITRE ATT&CK** Mapping (contextual awareness)

Why Integrate Wazuh with MISP

Challenge	Without MISP	With MISP
File alerts	Raw file events	Threat-context enriched
False positives	High	Reduced
SOC confidence	Low	High
Malware validation	Manual	Automated
Investigation speed	Slow	Faster

Benefit:

Only alerts backed by **global threat intelligence** are escalated.

**Architecture Overview****Flow:**

1. File created on endpoint
2. Wazuh Syscheck detects file change
3. Custom MISP integration script extracts file hashes
4. Hashes queried against MISP
5. If match found → enriched alert generated
6. Alert visible in Wazuh Dashboard
7. Alert forwarded to TheHive for SOC triage

► Step-by-Step Implementation**◆ Step 1: Enable File Integrity Monitoring (FIM)**

Purpose: Detect file creation/modification in high-risk directories.

- Enabled real-time monitoring on:
 - /tmp
 - /usr/bin
 - Windows user & temp directories
- Excluded system noise directories

✓ **Result:** Wazuh generates alerts when files are added or modified.

```
<syscheck>
  <disabled>no</disabled>

  <!-- High-risk directories only -->
  <directories check_all="yes" realtime="yes">/tmp</directories>
  <directories check_all="yes" realtime="yes">/usr/bin</directories>
  <directories check_all="yes" realtime="yes">/usr/local/bin</directories>

  <!-- Exclude noise -->
  <ignore>/var/log</ignore>
  <ignore>/proc</ignore>
  <ignore>/sys</ignore>
</syscheck>
```

```
<!-- File integrity monitoring -->
<syscheck>

  <disabled>no</disabled>

  <!-- Malware-prone user and temp locations -->
  <directories check_all="yes" realtime="yes">C:\Users</directories>
  <directories check_all="yes" realtime="yes">C:\Windows\Temp</directories>
  <directories check_all="yes" realtime="yes">C:\ProgramData</directories>
```

◆ Step 2: Develop Custom MISP Integration Script

Purpose: Query MISP for file hash reputation.

Actions performed:

- Created custom Python script:
 - custom-misp-file-hashes.py
- Script extracts:
 - MD5
 - SHA1
 - SHA256
- Queries MISP REST API
- Parses response and builds enriched JSON

✓ **Result:** Script successfully communicates with MISP and returns IOC data.

```
root@ip-1[REDACTED]:/var/ossec/integrations# nano custom-misp-file-hashes.py
root@ip-1[REDACTED]:/var/ossec/integrations# chmod 750 custom-misp-file-hashes.py
root@ip-1[REDACTED]:/var/ossec/integrations# chown root:wazuh custom-misp-file-hashes.py
root@ip-1[REDACTED]:/var/ossec/integrations#
```

```
GNU nano 7.2 custom-misp-file-hashes.py
#!/var/ossec/framework/python/bin/python3
# Copyright (C) 2025, CIRCL and Luciano Righetti
#
# This program is free software; you can redistribute it
# and/or modify it under the terms of the AGPL-3.0 license

import json
import os
import re
import sys
from socket import AF_UNIX, SOCK_DGRAM, socket

# Exit error codes
ERR_NO_REQUEST_MODULE = 1
ERR_BAD_ARGUMENTS = 2
ERR_BAD_HASHES = 3
ERR_NO_RESPONSE_MISP = 4
ERR_SOCKET_OPERATION = 5
ERR_FILE_NOT_FOUND = 6
ERR_INVALID_JSON = 7
```

◆ Step 3: Configure Wazuh Integration Block

Purpose: Tell Wazuh when to execute the MISP script.

Actions:

- Added integration block in ossec.conf
- Triggered only on **syscheck (file integrity) events**
- Enabled JSON alert format

✓ **Result:** Script runs automatically when file events occur.

```
<integration>
  <name>custom-misp-file-hashes.py</name>
  <hook_url>https://</hook_url>
  <api_key>f</api_key>
  <group>syscheck</group>
  <rule_id>554</rule_id>
  <alert_format>json</alert_format>
  <options>{
    "timeout": 10,
    "retries": 3,
    "debug": false,
    "tags": ["tlp:white", "tlp:clear", "malware"],
    "push_sightings": true,
    "sightings_source": "wazuh"
  }
</options>
</integration>
```

◆ Step 4: Create Custom Rules for Alert Refinement

Purpose: Convert enriched data into SOC-relevant alerts.

Actions:

- Created custom rule file
- Generated alert only when:
 - misp_file_hashes.found = 1
- Increased alert severity for confirmed threats

✓ Result: Clean, high-confidence alerts without noise.

```
GNU nano 7.2 misp_file_hashes.xml
<group name="misp,malware,">
  <rule id="100800" level="0">
    <decoded_as>json</decoded_as>
    <description>MISP: file hash check</description>
    <field name="integration">misp_file_hashes</field>
    <options>no_full_log</options>
  </rule>
  <rule id="100801" level="0">
    <if_sid>100800</if_sid>
    <field name="misp_file_hashes.found">0</field>
    <description>MISP: file hash not found</description>
  </rule>
  <rule id="100802" level="12">
    <if_sid>100800</if_sid>
    <field name="misp_file_hashes.found">1</field>
    <description>MISP: file hash matched</description>
  </rule>
  <rule id="100803" level="10">
    <if_sid>100800</if_sid>
    <field name="misp_file_hashes.error">403</field>
    <description>MISP ERROR: Invalid MISP credentials, check your the api_key setting configured
      in the MISP integration is valid MISP AuthKey</description>
```

```
root@ip-10-10-10-10: /var/ossec/etc/rules# systemctl restart wazuh-manager
root@ip-10-10-10-10: /var/ossec/etc/rules# grep -i misp-file /var/ossec/logs/ossec.log
2026/01/17 08:31:18 wazuh-integratord: INFO: Enabling integration for: 'custom-misp-file-hashes.py'.
2026/01/17 08:32:21 wazuh-integratord: INFO: Enabling integration for: 'custom-misp-file-hashes.py'.
root@ip-10-10-10-10: /var/ossec/etc/rules#
```

◆ Step 5: Validate Using Real Malware Test (EICAR)

Test Method:

- Downloaded EICAR test file on Ubuntu agent
- File created in monitored directory
- Hash matched existing MISP IOC

✔ Detection confirmed:

- Wazuh generated enriched alert
- MISP metadata attached
- Alert severity elevated

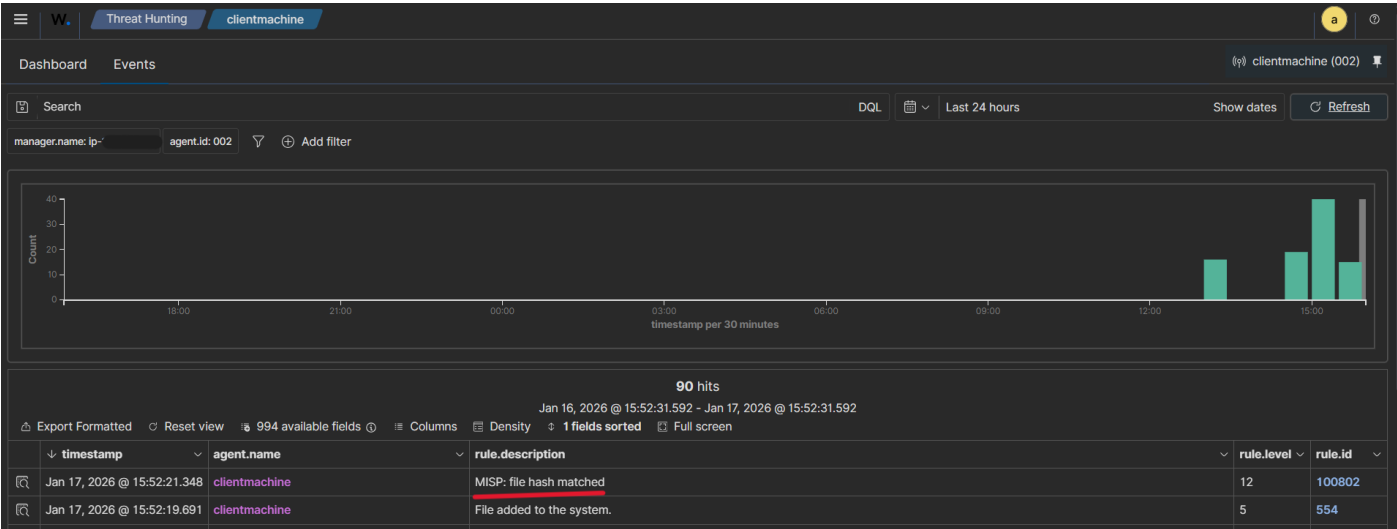
```
root@ip-...:/home/ubuntu# curl -L -o /tmp/eicar_final_test.exe https://secure.eicar.org/eicar.com
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100  68  100    68    0    0   121      0  --:--:-- --:--:-- --:--:--   122
root@ip-...:/home/ubuntu# md5sum /tmp/eicar_final_test.exe
44d88612fea8a8f36de82e1278abb02f /tmp/eicar_final_test.exe
root@ip-...:/home/ubuntu#
```

◆ Step 6: SOC Visibility in Wazuh Dashboard

Observed fields:

- integration: misp_file_hashes
- found: 1
- MISP event UUID
- Attribute UUID
- Permalink to MISP event

✔ SOC analysts can immediately verify threat context.



Document Details

[View surrounding documents](#)[View single document](#)

Table JSON

t _index	wazuh-alerts-4.x-2026.01.17
t agent.id	002
t agent.ip	
t agent.name	clientmachine
t data.integration	misp_file_hashes
② data.misp_file_hashes.attribute_uuid	7e719dba-ffdd-4bde-b121-bb6ac6d648d6
② data.misp_file_hashes.event_uuid	95ef6328-1530-4b5b-a3d8-40374b1a3e91
② data.misp_file_hashes.found	1
② data.misp_file_hashes.permalink	https://. /events/view/95ef6328-1530-4b5b-a3d8-40374b1a3e91
② data.misp_file_hashes.type	md5
② data.misp_file_hashes.value	44d88612fea8a8f36de82e1278abb02f
t decoder.name	json
t full_log	{"integration": "misp_file_hashes", "misp_file_hashes": {"found": 1, "type": "md5", "value": "44d88612fea8a8f36de82e1278abb02f", "event_uuid": "95ef6328-1530-4b5b-a3d8-40374b1a3e91", "attribute_uuid": "7e719dba-ffdd-4bde-b121-bb6ac6d648d6", "permalink": "https://35.173.143.187/events/view/95ef6328-1530-4b5b-a3d8-40374b1a3e91"}}}
t id	1768645341.1450416

◆ Step 7: Alert Forwarding to TheHive

Purpose: Incident triage and investigation readiness.

Result:

- Enriched Wazuh alerts visible in **TheHive Alerts**
- Observables automatically attached
- Analyst can manually convert alert into case if required

✓ **Maintains** analyst control while improving investigation speed.

Alerts / wazuh_alert (#75fc60) / Description

Enter a case number + Create Case

→ MISP: file hash matched

id ~184668248
Created by apianalyst
Created at 17/01/2026 18:44

SEVERITY:MEDIUM
TLP:AMBER PAP:AMBER

Assignee Assign to me
Unassigned

Source
wazuh

Reference
75fc60

Type
wazuh_alert

Occurred date
17/01/2026 18:44

Status
New

Time metrics
Detection
< 1 second

General Observables (7) TTPs (0) Attachments Similar Cases Similar Alerts Responders History

* Title
MISP: file hash matched

Tags
agent_name=clientmachine rule=100802 wazuh agent_ip= agent_id=002

Description

Timestamp

key	val
timestamp	2026-01-17T13:14:12.335+0000

Rule

key	val
rule.level	12
rule.description	MISP: file hash matched
rule.id	100802
rule.firedtimes	1
rule.mail	True

Project Outcome

- Successfully implemented **threat-intelligence-driven malware detection**
- Reduced false positives by correlating with MISP
- Demonstrated real-world SOC enrichment workflow
- Validated detection using live malware test
- Maintained clean alert pipeline without automation overload

Skills Gained

- SIEM integration engineering
- Threat intelligence correlation
- Python scripting for SOC automation
- Wazuh rule tuning and alert refinement
- MISP IOC lifecycle understanding
- SOC investigation workflow design
- TheHive alert triage fundamentals

Conclusion

This project showcases a **practical SOC-grade implementation** of threat intelligence enrichment using Wazuh and MISP.

Rather than generating noisy alerts, the system produces **high-confidence detections backed by global intelligence**, making it suitable for real SOC environments and security operations roles.